PRAKTIKUM SISTEM OPERASI

MODUL VI

PROSES SEKUENSIAL DAN KONKUREN

Oleh:

Marvel Sanjaya Setiawan 2311104053

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

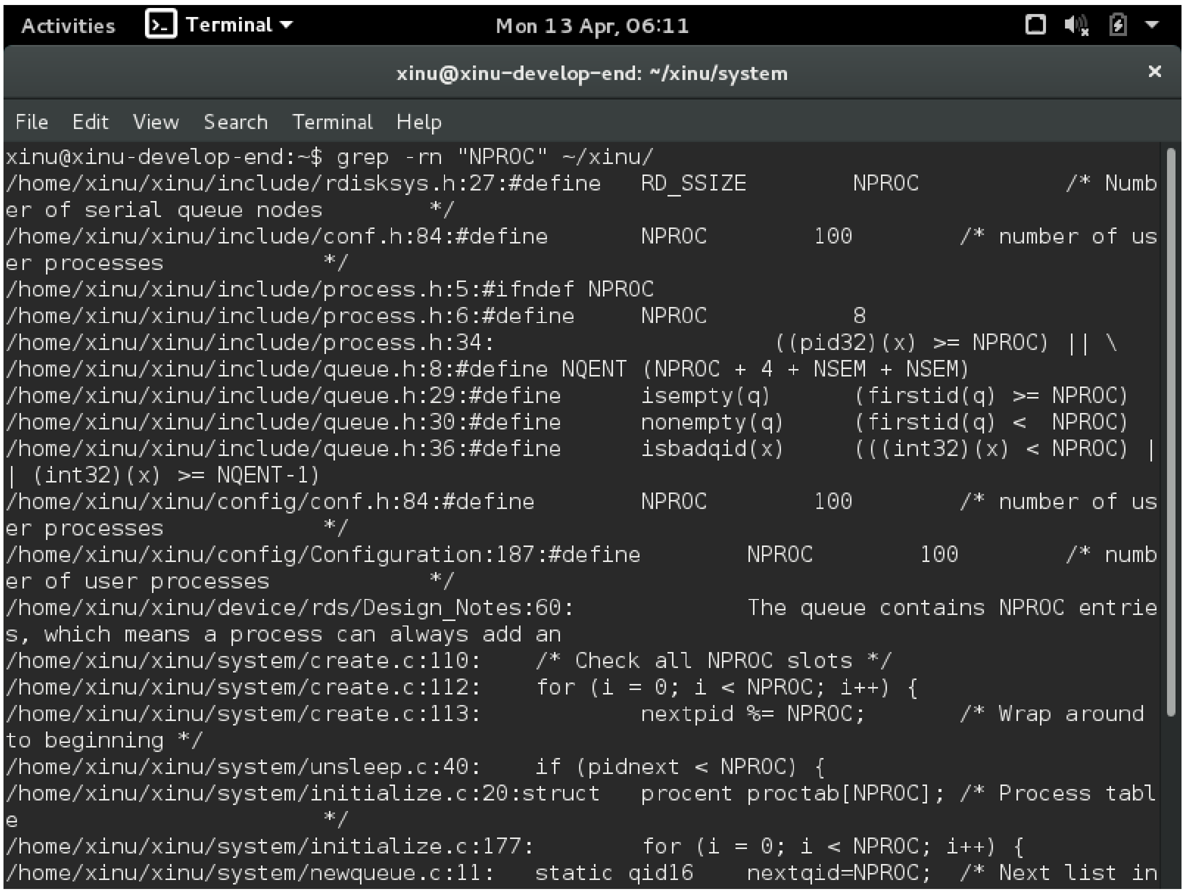
UNIVERSITAS TELKOM

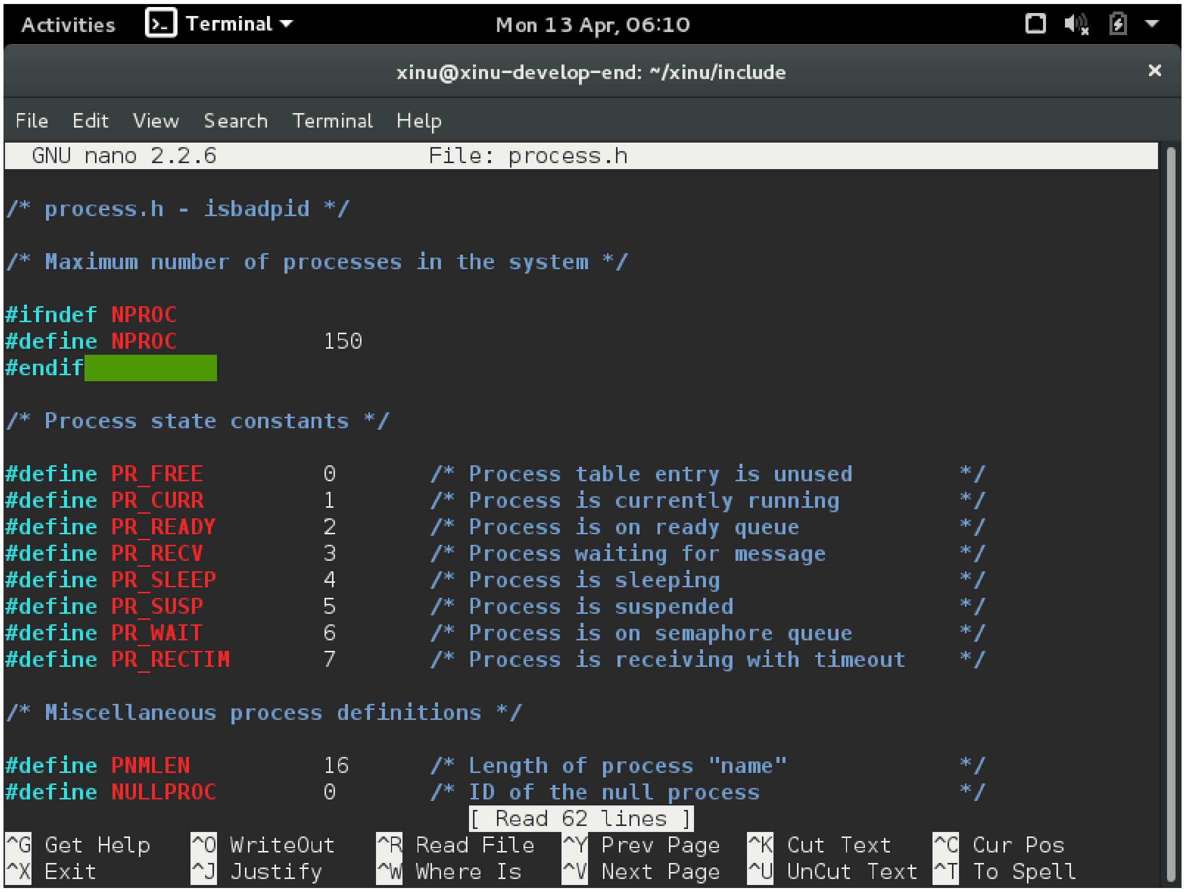
2026

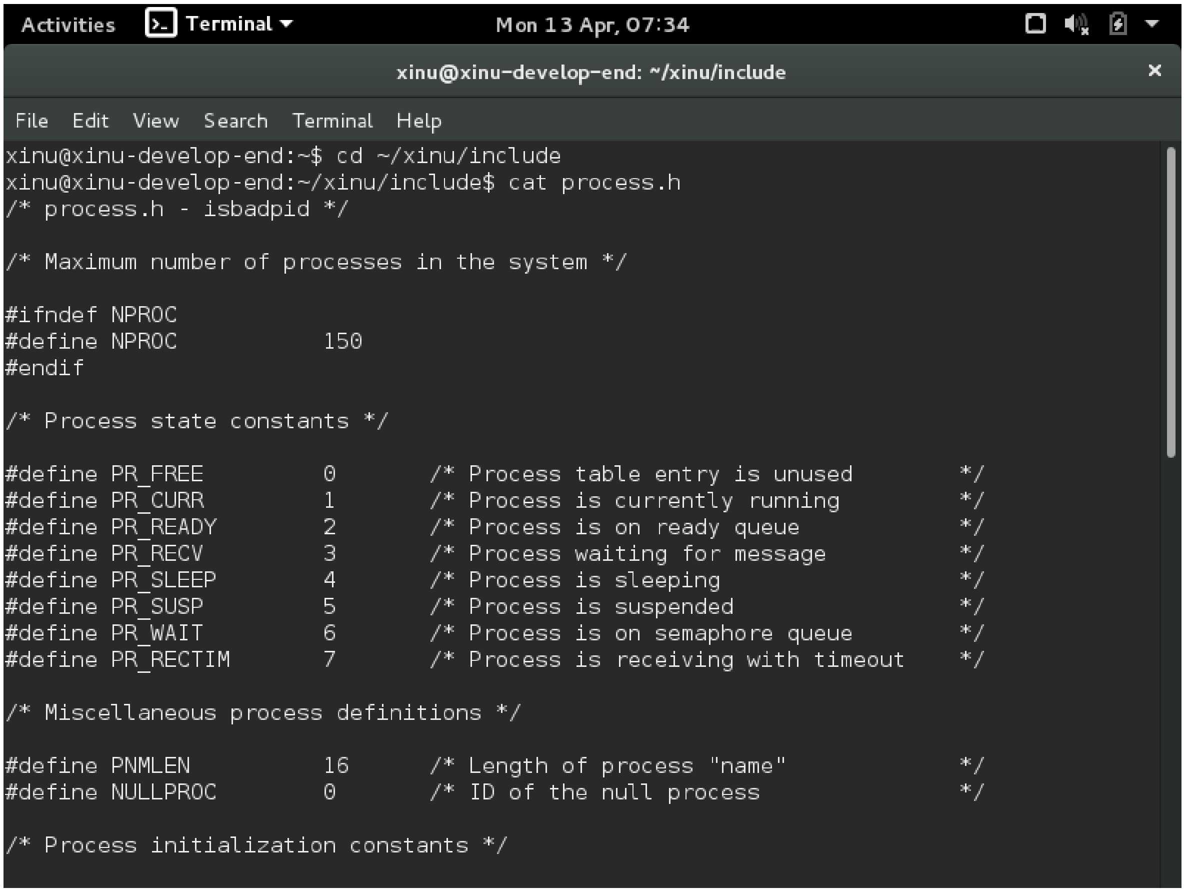
JURNAL

1. **Konfigurasi Batas Proses: Cari di source code Xinu bagian yang mengatur jumlah maksimal proses. Sebutkan nilai defaultnya, lalu ubah batas tersebut menjadi 150.**

   Jawaban:



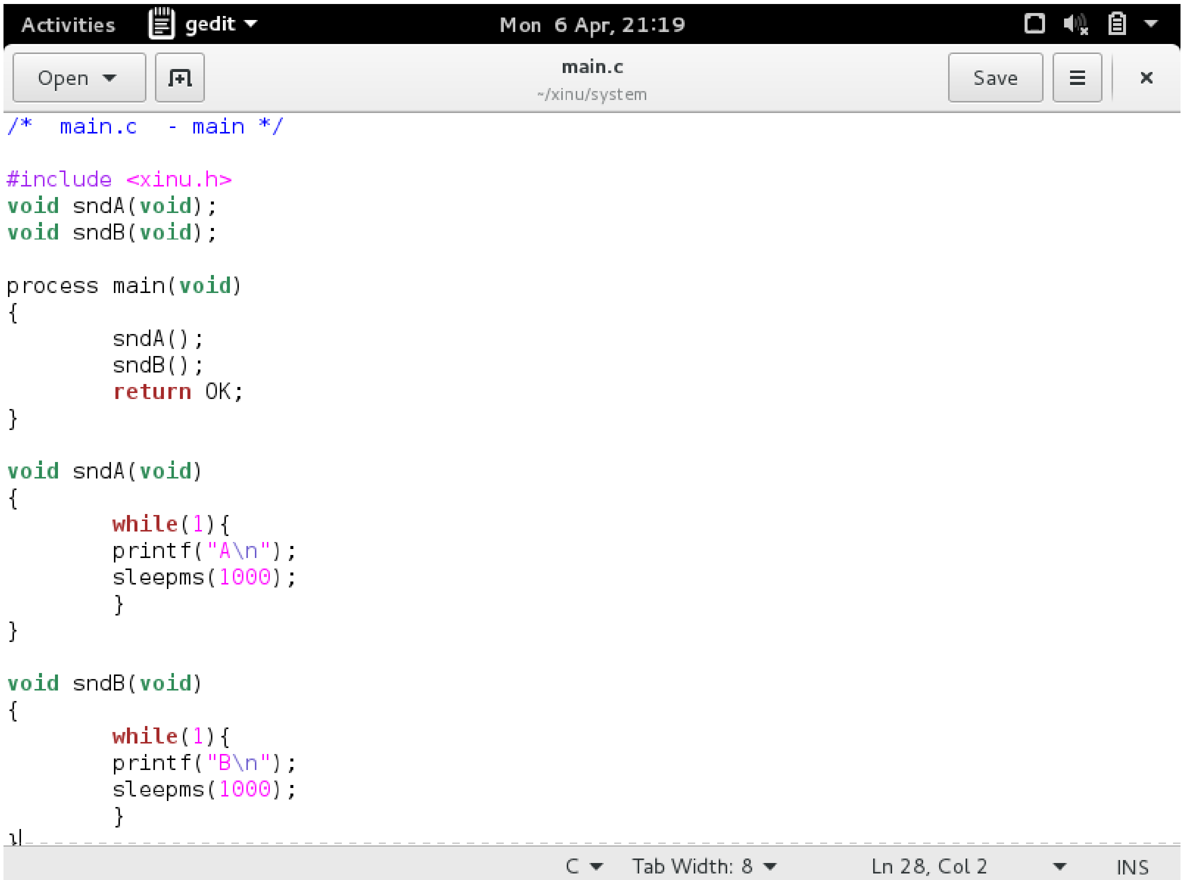

Activities
Terminal
Mon 13 Apr, 06:10
xinu@xinu-develop-end: ~/xinu/include
File Edit View Search Terminal Help
GNU nano 2.2.6 File: process.h
/* process.h - isbadpid */
/* Maximum number of processes in the system */
#ifndef NPROC
#define NPROC 150
#endif
/* Process state constants */
#define PR_FREE 0 /* Process table entry is unused */
#define PR_CURR 1 /* Process is currently running */
#define PR_READY 2 /* Process is on ready queue */
#define PR_RECV 3 /* Process waiting for message */
#define PR_SLEEP 4 /* Process is sleeping */
#define PR_SUSP 5 /* Process is suspended */
#define PR_WAIT 6 /* Process is on semaphore queue */
#define PR_RECTIM 7 /* Process is receiving with timeout */
/* Miscellaneous process definitions */
#define PNMLEN 16 /* Length of process "name" */
#define NULLPROC 0 /* ID of the null process */
[Read 62 lines]
^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where Is ^V Next Page ^U UnCut Text ^T To Spell

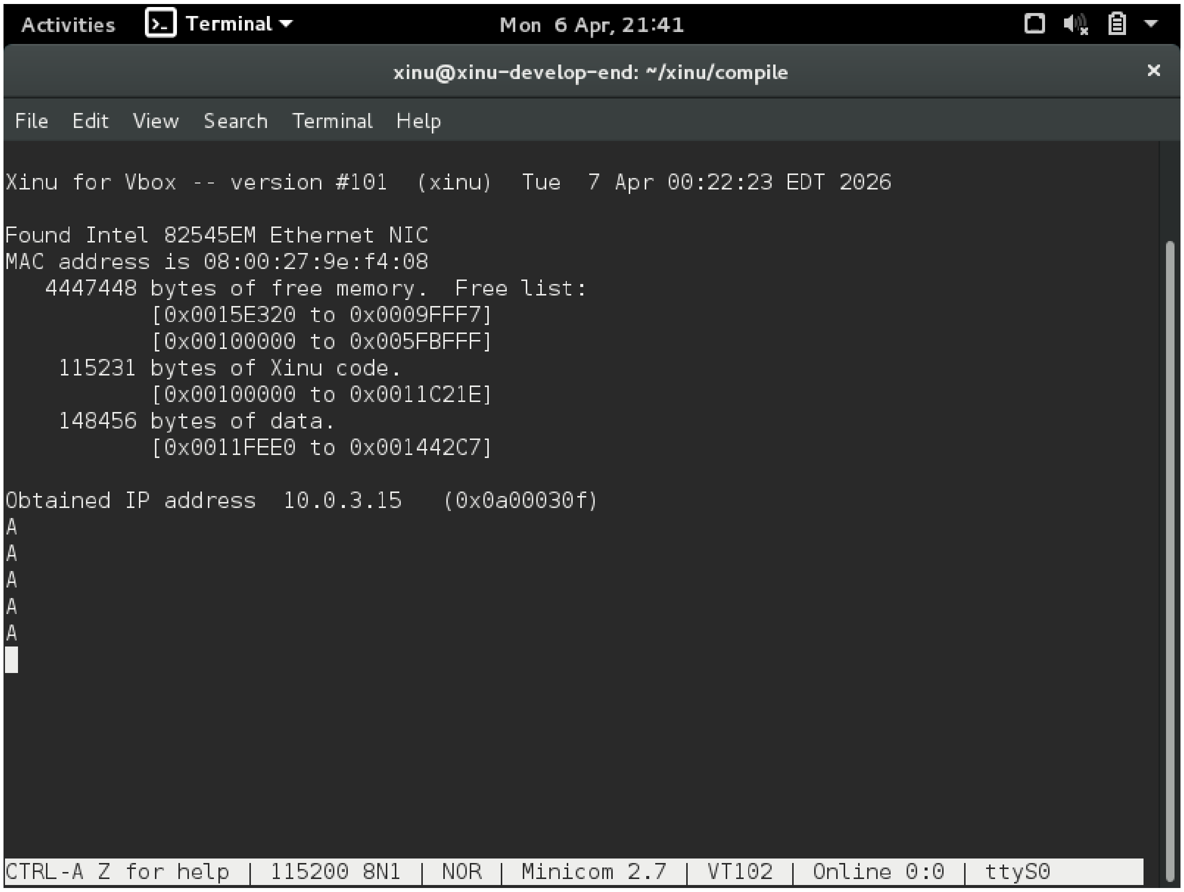


a. **Cari file process.h dengan perintah: grep -rn "NPROC" ~/xinu/**
b. **Masuk ke folder include: cd ~/xinu/include**
c. **Edit file process.h: nano process.h**
d. **Cari baris #define NPROC 8, ubah angka 8 → 150**
   Simpan: Ctrl+O → Enter → Ctrl+X
e. **Verifikasi:**
f. **cat process.h**

2. Run: main_ex1

Jawaban:





Urutan pengerjaan:

1. **Masuk ke folder system: cd ~/xinu/system**
2. **Edit main.c: gedit main.c**
3. **Isi kode:**

```
/* main.c - main */
  #include <xinu.h>
  void sndA(void);
  void sndB(void);

  process main(void) {
     sndA();
     sndB();
     return OK;
  }
  void sndA(void) {
     while(1) {
        printf("A\n");
        sleepms(1000);
```

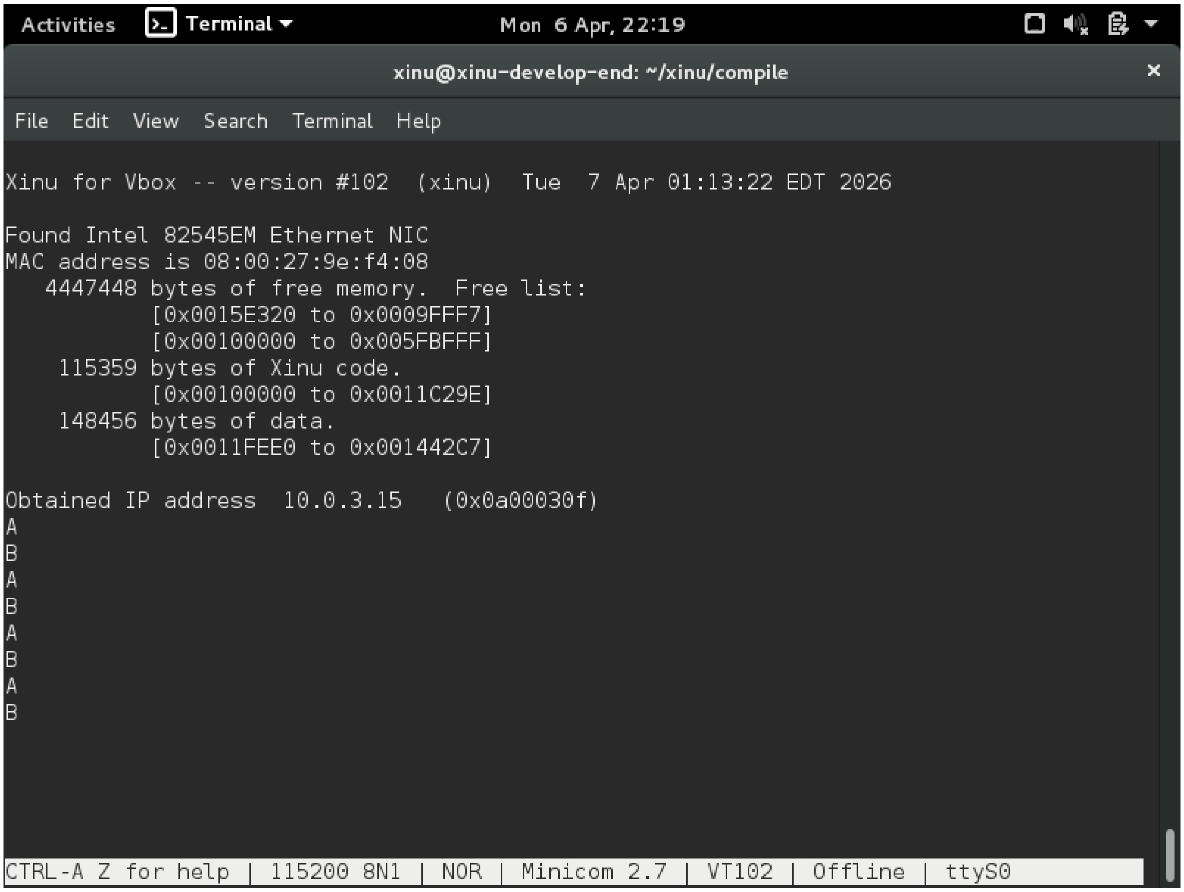
```
    }
}
void sndB(void) {
    while(1) {
        printf("B\n");
        sleepms(1000);
    }
}
```

4. **Simpan, lalu compile: cd ~/xinu/compile && make**
5. **Jalankan minicom: sudo minicom**

3. **Run: main_ex2**

Jawaban:


```c
/*  main.c  - main */

#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
	resume(create(sndA, 1024, 20, "process A", 0));
	resume(create(sndB, 1024, 20, "process B", 0));
	return OK;
}

void sndA(void)
{
	while(1){
	printf("A\n");
	sleepms(1000);
	}
}

void sndB(void)
{
	while(1){
	printf("B\n");
	sleepms(1000);
	}
}
```



Urutan pengerjaan:

1. **Edit kembali ~/xinu/system/main.c:**

```
/* main.c - main */
#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void) {
   resume(create(sndA, 1024, 20, "process A", 0));
   resume(create(sndB, 1024, 20, "process B", 0));
   return OK;
}
void sndA(void) {
   while(1) {
      printf("A\n");
      sleepms(1000);
   }
}
```

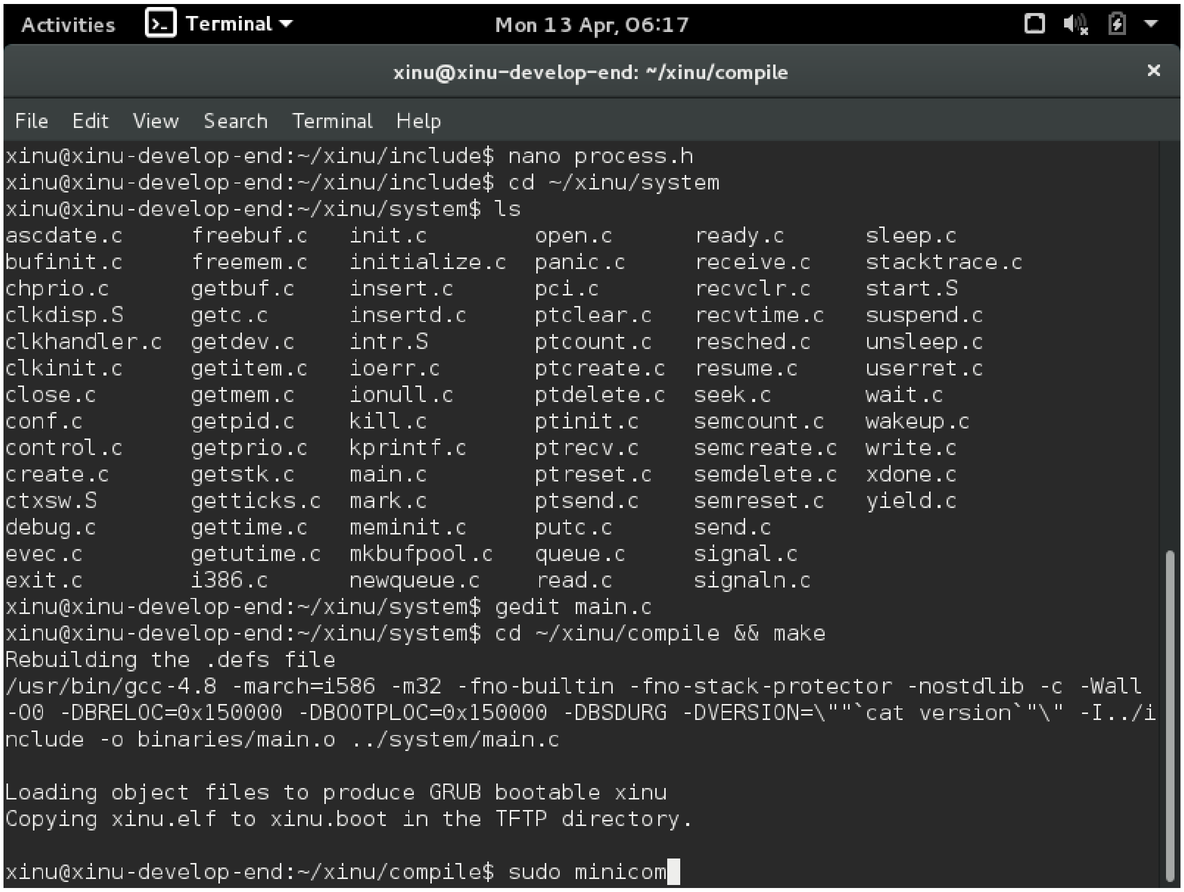
```
void sndB(void) {
   while(1) {
      printf("B\n");
      sleepms(1000);
   }
}
```

2. **Simpan → compile → jalankan minicom**

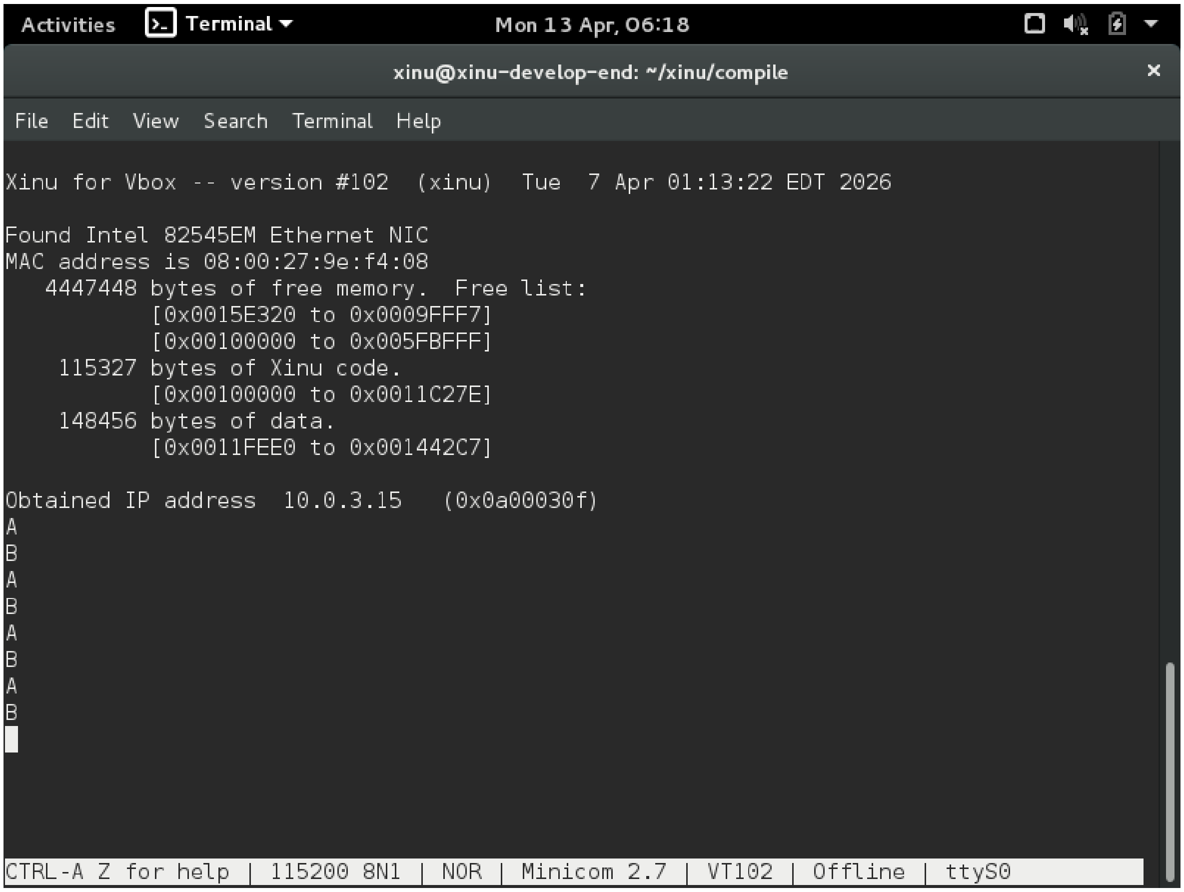
4. **Run: main_ex3**

Jawaban:


```c
/*  main.c  - main */

#include <xinu.h>
void sndChar(char);

process main(void)
{
	resume(create(sndChar, 1024, 20, "send A", 1, 'A'));
	resume(create(sndChar, 1024, 20, "send B", 1, 'B'));
	return OK;
}

void sndChar(char c)
{
	while(1){
	printf("%c \n", c);
	sleepms(1000);
	}
}
```



```
Activities    Terminal ▾    Mon 13 Apr, 06:17
xinu@xinu-develop-end: ~/xinu/compile
File  Edit  View  Search  Terminal  Help
xinu@xinu-develop-end:~/xinu/include$ nano process.h
xinu@xinu-develop-end:~/xinu/include$ cd ~/xinu/system
xinu@xinu-develop-end:~/xinu/system$ ls
ascdate.c     freebuf.c   init.c        open.c      ready.c      sleep.c
bufinit.c     freemem.c   initialize.c  panic.c     receive.c    stacktrace.c
chprio.c      getbuf.c    insert.c      pci.c       recvclr.c    start.S
clkdisp.S     getc.c      insertd.c     ptclear.c   recvtime.c   suspend.c
clkhandler.c  getdev.c    intr.S        ptcount.c   resched.c    unsleep.c
clkinit.c     getitem.c   ioerr.c       ptcreate.c  resume.c     userret.c
close.c       getmem.c    ionull.c      ptdelete.c  seek.c       wait.c
conf.c        getpid.c    kill.c        ptinit.c    semcount.c   wakeup.c
control.c     getprio.c   kprintf.c     ptrecv.c    semcreate.c  write.c
create.c      getstk.c    main.c        ptreset.c   semdelete.c  xdone.c
ctxsw.S       getticks.c  mark.c        ptsend.c    semreset.c   yield.c
debug.c       gettime.c   meminit.c     putc.c      send.c
evec.c        getutime.c  mkbufpool.c   queue.c     signal.c
exit.c        i386.c      newqueue.c    read.c      signaln.c
xinu@xinu-develop-end:~/xinu/system$ gedit main.c
xinu@xinu-develop-end:~/xinu/system$ cd ~/xinu/compile && make
Rebuilding the .defs file
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/main.o ../system/main.c

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.

xinu@xinu-develop-end:~/xinu/compile$ sudo minicom
```



Urutan pengerjaan:

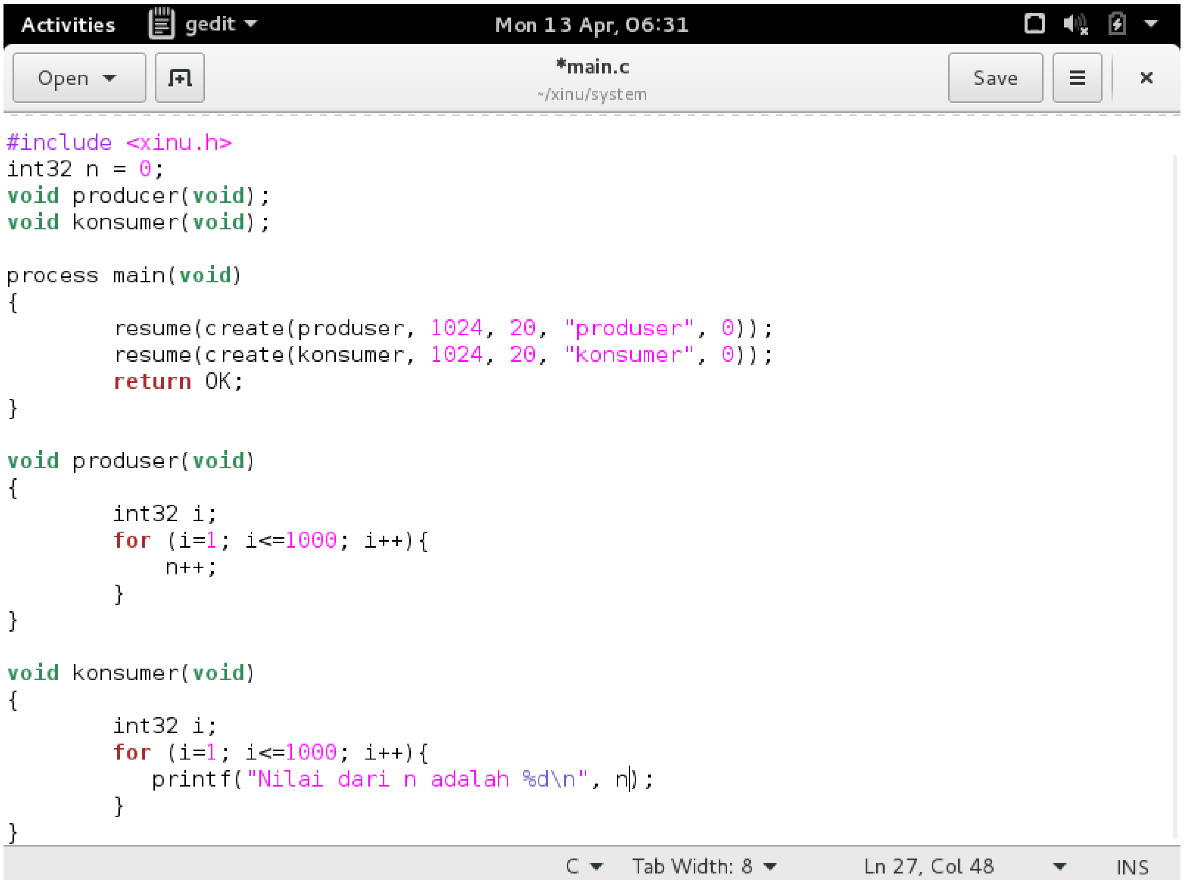
1. **Edit main.c:**

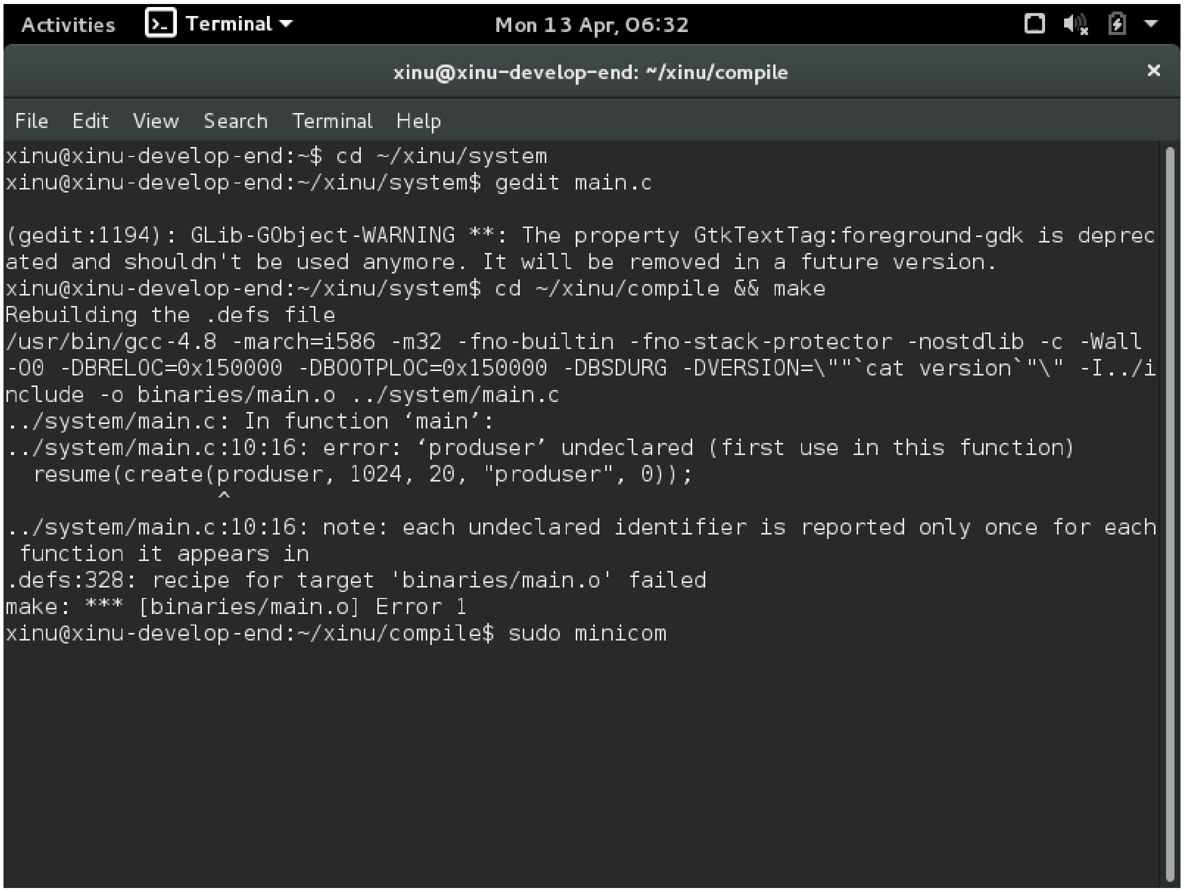
```
/* main.c - main */
#include <xinu.h>
void sndChar(char);

process main(void) {
   resume(create(sndChar, 1024, 20, "send A", 1, 'A'));
   resume(create(sndChar, 1024, 20, "send B", 1, 'B'));
   return OK;
}
void sndChar(char c) {
   while(1) {
      printf("%c \n", c);
      sleepms(1000);
   }
}
```

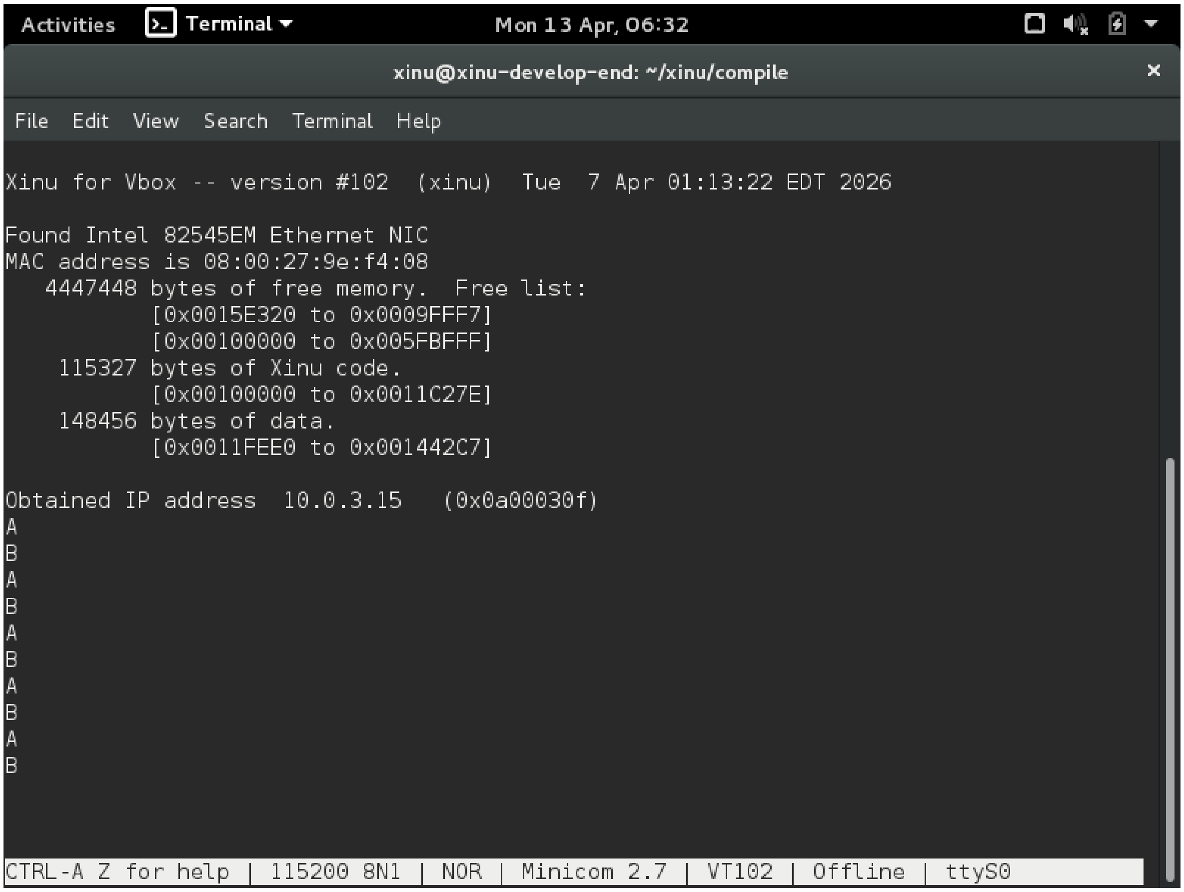
2. Simpan → compile → jalankan minicom

5. **Analisis Sinkronisasi: Buat proses Producer (isi variabel n 1-1000) dan Consumer (cetak variabel n). Jelaskan mengapa outputnya tidak sesuai urutan/ekspektasi (analisis race condition).**

Jawaban:




Activities
Terminal
Mon 13 Apr, 06:32
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
xinu@xinu-develop-end:~$ cd ~/xinu/system
xinu@xinu-develop-end:~/xinu/system$ gedit main.c
(gedit:1194): GLib-GObject-WARNING **: The property GtkTextTag:foreground-gdk is deprec
ated and shouldn't be used anymore. It will be removed in a future version.
xinu@xinu-develop-end:~/xinu/system$ cd ~/xinu/compile && make
Rebuilding the .defs file
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-O0 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=\""`cat version`"\" -I../i
nclude -o binaries/main.o ../system/main.c
../system/main.c: In function ‘main’:
../system/main.c:10:16: error: ‘produser’ undeclared (first use in this function)
resume(create(produser, 1024, 20, "produser", 0));
^
../system/main.c:10:16: note: each undeclared identifier is reported only once for each
function it appears in
.defs:328: recipe for target 'binaries/main.o' failed
make: *** [binaries/main.o] Error 1
xinu@xinu-develop-end:~/xinu/compile$ sudo minicom



Urutan pengerjaan:

1. **Edit main.c:**

```
/* main.c - main */
/* Nama: [nama kamu] - NIM: [nim kamu] */
#include <xinu.h>
int32 n = 0;
void produser(void);
void konsumer(void);

process main(void) {
   resume(create(produser, 1024, 20, "produser", 0));
   resume(create(konsumer, 1024, 20, "konsumer", 0));
   return OK;
}
void produser(void) {
   int32 i;
   for (i=1; i<=1000; i++) {
      n++;
```

```
    }
  }
  void konsumer(void) {
    int32 i;
    for (i=1; i<=1000; i++) {
      printf("Nilai dari n adalah %d\n", n);
    }
  }
```

2. **Simpan → compile → jalankan minicom**
3. **Screenshot output**
4. **Tulis penjelasan: Kenapa hasilnya langsung 1000 semua?**
   Karena kedua proses berjalan konkuren, ada kemungkinan proses produser selesai duluan melakukan loop 1000 kali (sehingga n = 1000) sebelum konsumer sempat membaca nilai n yang kecil. Akibatnya, saat konsumer mulai membaca, nilai n sudah = 1000 dan semua output menampilkan 1000. Inilah yang disebut race condition — urutan eksekusi proses tidak bisa diprediksi tanpa sinkronisasi.